

ДЕЛОВИ КОМПАЈЛЕРА

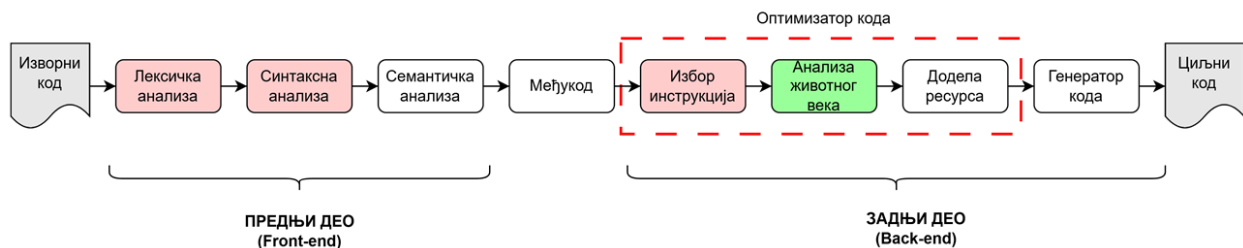
Анализа животног века

Процесор располаже ограниченим бројем регистара, па није могуће да свака променљива има свој засебан регистар током извршавања програма. Уколико знамо у ком делу програма се вредност променљиве више не користи, њен регистар можемо доделити другој променљивој без утицаја на исправност извршавања.

Да би се омогућило да различите променљиве које нису истовремено у употреби деле исти ресурс, неопходна је информација о животном веку променљиве. Променљива је „жива“ ако ће се њена вредност користити у будућности. Ако се више не користи, њен ресурс можемо поново употребити.

Животни век променљиве започиње дефиницијом променљиве (упис почетне вредности), а завршава се последњом употребом (последње читавање садржаја променљиве).

Анализа животног века (енгл. *liveness analysis*) је анализа тока података која за сваку тачку програма одређује да ли је вредност неке променљиве у тој тачки потребна у наставку извршавања. Ова анализа је једна од кључних анализа у компајлеру, јер директно утиче на алокацију регистара. Ако знамо у којим деловима програма је променљива „жива“, можемо одредити да ли различите променљиве могу делити исти регистар.

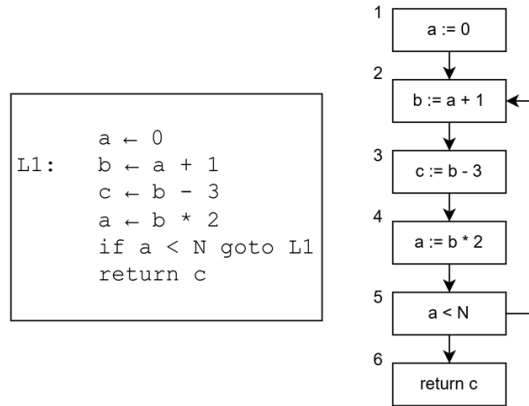


Слика 1: Основне фазе компајлера (преводиоца)

Први корак у анализи животног века је конструисање **графа тока управљања** који садржи:

- **Чворове** – инструкције или блок кода,
- **Гране** – ток управљања програмом.

Граф тока управљања се конструише тако што се за сваку инструкцију дефинише **чвор** графа, а за свака два чвора x и y (где x претходи y) повлачи се **стрелица** од x ка y .



Слика 2: Пример графа тока управљања

За сваки чвор (инструкцију) дефинишу се следећи **скупови**:

1. **Скуп претходника** $\text{pred}[n]$ – чворови који долазе из улазне стрелице,
2. **Скуп наследника** $\text{succ}[n]$ – чворови у које улази стрелица,
3. **Скуп променљивих које чвор користи** $\text{use}[n]$ – исто што и *source*,
4. **Скуп променљивих које чвор дефинише** $\text{def}[n]$ – исто што и *destination*.

Табела 1 приказује скупове претходника, наследника, коришћених и дефинисаних променљивих за пример са Сlike 2, који има 6 чворова.

Табела 1: Приказ скупова претходника, наследника, коришћених и дефинисаних променљивих из примера

ЧВОР	Инструкција – чвор	$\text{pred}[n]$	$\text{succ}[n]$	$\text{use}[n]$	$\text{def}[n]$
1	<code>a := 0</code>	–	{2}	–	{a}
2	<code>b := a + 1</code>	{1, 5}	{3}	{a}	{b}
3	<code>c := b - 3</code>	{2}	{4}	{b}	{c}
4	<code>a := b * 2</code>	{3}	{5}	{b}	{a}
5	<code>a < N</code>	{4}	{2, 6}	{a}	–
6	<code>return C</code>	{5}	–	{c}	–

Променљива је жива:

- **на стрелици графа** – ако постоји усмерена путања од те стрелице до чвора који користи ту променљиву, а да путања не прелази преко чвора који дефинише ту исту променљиву.
- **на улазу чвора** – ако је жива на било којој улазној стрелици чвора (скуп $\text{in}[n]$),
- **на излазу чвора** – ако је жива на било којој излазној стрелици чвора (скуп $\text{out}[n]$).

Информација о животу променљивих се може изразити на основу скупова $\text{def}[n]$ и $\text{use}[n]$.

Резултат анализе су следећи скупови:

- **$\text{in}[n]$** — променљиве живе на улазу у чвор n , тј. скуп променљивих које морају бити доступне пре извршавања чвора n ,
- **$\text{out}[n]$** — променљиве живе на излазу из чвора n , тј. скуп променљивих које морају бити сачуване након извршавања чвора n , јер ће бити потребне у неком од наредних чворова.

Анализа животног века је анализа која се реализује **уназад** (од последњег до првог чвора), јер на тај начин одређујемо које вредности ће нам бити потребне у будућности.

Алгоритам анализе животног века променљивих

Први корак у анализи животног века је одређивање in и out скупова. Скуп $out[n]$ је унија „ in “ скупова свих наследника чвора n , односно унија свих променљивих s које припадају скупу „ in “ сваког наследника чвора n . Скуп $in[n]$ чине све променљиве из скупа $use[n]$ плус променљиве које јесу у $out[n]$, али нису у $def[n]$. Одатле следе једначине тока података за анализу животног века променљивих:

$$\begin{aligned} out[n] &\leftarrow \bigcup_{s \in succ[n]} in[s] \\ in[n] &\leftarrow use[n] \cup (out[n] - def[n]) \end{aligned}$$

Решење ових једначина се добија следећим алгоритмом, исказаним у псеудокоду:

1. Иницијализација: За све чворове:
 - $in[n] = \emptyset$
 - $out[n] = \emptyset$
2. Понављање: Итерира се кроз чворове (најчешће од последњег ка првом) за сваки чвор n :
 - $out[n] = \text{унија } in[s] \text{ за све } s \in succ[n]$
 - $in[n] = use[n] \cup (out[n] - def[n])$
3. Услов заустављања:
 - Алгоритам се понавља све док скупови 2 узастопне итерације не постану једнаки.

```
for each n
    in[n] ← {}; out[n] ← {}
repeat
    for each n
        in'[n] ← in[n]; out'[n] ← out[n]1
        out[n] ←  $\bigcup_{s \in succ[n]} in[s]$ 
        in[n] ←  $use[n] \cup (out[n] - def[n])$ 
until in'[n] = in[n] and out'[n] = out[n] for all n
```

Петља псеудокода „**for each** n “ означава да се тело петље мора извршити за свако n (у овом случају за сваку инструкцију, тј. чвор графа тока), али не одеђује којим редоследом. И заиста, да би се добило решење једначина животног века није важно којим редоследом ће унутрашња петља итерирати кроз инструкције. Међутим, брзина проналажења решења ипак зависи од редоследа итерирања. Показује се да се до решења долази у мање итерација спољашње петље уколико унутрашња петља итерира од последње инструкције ка првој, него од прве ка последњој.

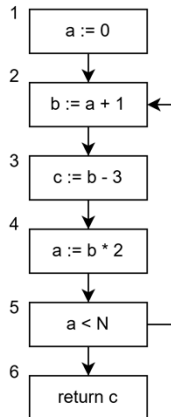
Питање: Шта се дешава ако унутрашња петља итерира од прве ка последњој инструкцији? Урадити ручно неколико првих рачунања за пример са Сlike 2.

За пример са Сlike 2 дати алгоритам са петљом организованом од последњег чвора (6) ка првом чвору (1) доводи до решења једначина у три итерације. Итерације се понављају све док 2 итерације in и out скупова не буду једнаке. Резултати за сваку итерацију су дати у Табели 2.

¹ Листе in' и out' су копије листа in и out на почетку сваке итерације (унутрашња **for** петља).

Прво се одређују наследници (succ), коришћене (use) и дефинисане (def) променљиве за сваки чвор, а затим се реализују итерације рачунања out и in скупова за сваку инструкцију. Out скуп представља унију свих in скупова наследника дате инструкције, док in скуп представља унију use скупа и разлике скупова (out - def). Итерације се понављају док out и in скупови не буду једнаки у 2 узастопне итерације.

Табела 2: Приказ итерација алгорита животног века променљивих a, b и c из примера



	succ	use	def	I		II		III	
				out	in	out	in	out	in
6	-	c	-	-	c	-	c	-	c
5	2, 6	a	-	c	ac	ac	ac	ac	ac
4	5	b	a	ac	bc	ac	bc	ac	bc
3	4	b	c	bc	b	bc	b	bc	b
2	3	a	b	b	a	b	a	b	a
1	2	-	a	a	-	a	-	a	-

Питање: Која комбинација променљивих не постоји у итерацијама алгорита? Шта то значи? Упоредите слику и табелу и проверите да ли су променљиве живе на стрелици, на улазу и на излазу у завршној итерацији.

Две променљиве које су истовремено у истом in или out скупу не могу делити исти регистар. То значи да променљиве a и c, као и b и c не могу делити исти регистар, док променљиве a и b могу.

Реализација алгорита за анализу животног века

Приликом реализације задатака је потребно користити постојеће структуре и функције. Једна инструкција описана је структуром `Instruction`. Она садржи тип инструкције (`type`), листе претходних и наредних инструкција `pred` и `succ`, листе променљивих које се користе и дефинишу: `use` и `def`, као и `in` и `out` листе, које представљају излаз из фазе анализе животног века променљивих. Више инструкција описује се као листа `Instructions` која се у функцији `makeExample()` попуњава примером кода. Променљиве су описане у структури `Variable` и садрже назив (`name`) и позицију (`pos`) као једина поља од интереса за ову вежбу. Више променљивих описано је помоћу листе `Variables`.

Типови и функције од значаја из библиотеке *libLivenessAnalysis.lib*:

```
typedef std::list<Variable*> Variables;           // листа променљивих

bool variableExists(Variable* variable, Variables variables);
// провера постојања променљиве „variable“ у листи „variables“

typedef std::list<Instruction*> Instructions;     // листа инструкција
```

Све операције могу се извести постојећим методама листе (`List`) из библиотеке стандардних шаблона (STL – Standard Template Library): **insert** (додавање једног елемента у листу или додавање дела друге листе), **sort** (сортирање листе), **unique** (брисање елемената који нису јединствени из **сортиране** листе), и **push_back** (додавање елемента на крај листе). Плитка копија листе се може једноставно направити коришћењем оператора доделе (`=`). Поређење једнакости две листе уграђених типова (у овом случају показивача) може се урадити коришћењем уобичајених оператора за поређење (`==`, `!=`).

За пример се узима листа инструкција са Сlike 2. Она садржи 3 променљиве `a`, `b` и `c` и 6 инструкција (чворова). Попуњавање листе инструкција по датом примеру је реализовано у функцији `makeExample()`.

ЗАДАТАК

Реализовати део алгорита за одређивање животног века променљивих. Потребно је реализовати **само унутрашњу петљу** из алгорита, ону која у обрнутом редоследу итерира кроз инструкције. Резултат упоредити са колоном I у Табели 2. Код писати у телу функције *livenessAnalysis*, чије заглавље је дато у датотеци *livenessAnalysis.h*, а празна дефиниција у датотеци *livenessAnalysis.cpp*. Реализовати места у коду која су обележена са `TO DO`.

ДОДАТНИ ЗАДАТАК

Реализовати комплетан алгорита за одређивање животног века променљивих. Резултат упоредити са колоном III у Табели 2.